

VOUCH

A Cross-Chain Loyalty Primitive

Exploring a New Primitive with Attestcoin: proof-gated loyalty state that moves across chains without ever moving an asset across a bridge.

PAYMENT CHAIN	Ethereum Sepolia · chain id 11155111
REWARD CHAIN	Creditcoin CC3 testnet · chain id 102031
TRUST LAYER	Attestcoin Protocol — native precompiles, no asset bridge
PROJECT	github.com/dun999/vouch
STATUS	Testnet hackathon prototype — demo denominations only
DOCUMENT	v1.1 · September 2026

This document describes a testnet hackathon prototype. MockUSDC is an openly mintable demo token, not redeemable USDC; dollar figures throughout are demo denominations. Nothing in this document is financial, investment, or legal advice.

Abstract

Loyalty programs are normally scoped to one merchant's database: a customer's history and rewards evaporate the moment they shop somewhere else, and every merchant rebuilds the same retention machinery from scratch. Vouch is a testnet prototype exploring a different design — let a purchase made on one chain become independently verifiable evidence on another, and build a shared loyalty primitive on top of that evidence, without ever moving an asset across a bridge.

Customers pay merchants in MockUSDC on Ethereum Sepolia. Attestcoin, a native attestation protocol built into Creditcoin, proves that the payment happened and lets Creditcoin-side contracts react to it directly. Vouch turns that proof into receipts, stars, levels, quest completions, and CTC cashback, recorded on a non-transferable, on-chain-rendered Benefit Pass. This paper describes the model, the mechanics of the Attestcoin integration that makes it possible, the incentive design layered on top of it, and what has — and has not yet — been validated against live infrastructure.

Contents

1. The Problem: Loyalty That Doesn't Travel	2
2. The Vouch Model	3
What customers and merchants can do	3
3. Attestcoin as the Trust Primitive	4
How a single purchase becomes a reward	4
The integration in detail	5
Two design decisions the proof format forces	5
Extending the same primitive to price data	6
4. Incentive Design	6
5. Contracts and Technology Stack	7
6. Security, Testing, and Live Validation	8
Live proof-to-reward validation	8
7. Known Limitations	9
8. Roadmap	9
Appendix A. Current Testnet Deployment	10
Appendix B. Attestcoin Protocol Constants	10
Appendix C. Earlier ETH-Payment Deployment	11

1. The Problem: Loyalty That Doesn't Travel

Most loyalty programs are scoped to a single merchant. A customer's purchase history, streaks, and rewards live inside one company's database, so every new merchant relationship starts from zero — regardless of how loyal or high-value that customer has been elsewhere. Merchants, in turn, each build and maintain their own retention systems to court the same overlapping pool of customers.

At the same time, merchants routinely spend meaningful marketing budgets chasing attention and clicks, a large share of which never converts into a purchase. Vouch's starting premise is that some of that spend can be redirected more directly: toward customers who have already completed a verifiable purchase, rather than toward impressions that may or may not.

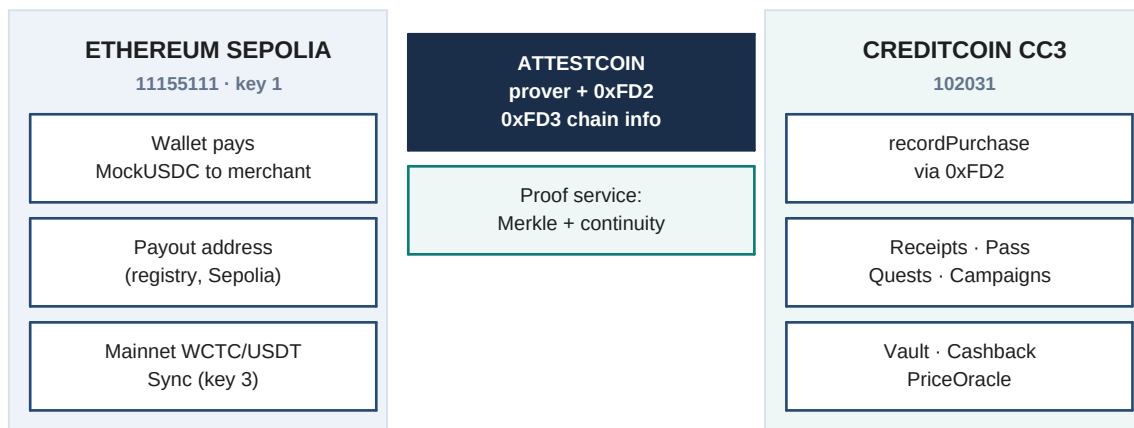
In practice this looks like a coffee shop rewarding a customer's repeat Flat White purchases, a bookstore rewarding a first-time purchase, or a ramen shop funding a community campaign that any customer can contribute toward. What makes these examples interoperable rather than one-off is that the customer carries a single loyalty pass across all of them, while every merchant's rewards remain independently funded and verified.

Building that shared pass requires a way to prove — cheaply, and without trusting any single company — that a purchase on one chain actually happened, so that a different chain entirely can react to it. That is the role Attestcoin plays in Vouch, and the subject of Section 3.

2. The Vouch Model

Vouch is a testnet prototype for a cross-chain loyalty app. Customers pay merchants in MockUSDC on Ethereum Sepolia; Attestcoin proves that payment to Creditcoin; and Vouch's contracts on Creditcoin CC3 testnet turn that proof into receipts, stars, levels, quest progress, and CTC cashback. A customer's progress lives in a non-transferable Benefit Pass — an ERC-721 token whose artwork is generated entirely on-chain as SVG, rather than hosted off-chain.

The two chains play distinct, non-overlapping roles. Payments happen — and stay — on Sepolia; loyalty state and reward accounting happen — and stay — on Creditcoin. Nothing is bridged between them: no wrapped token, no locked collateral, no custodial relayer holding funds in escrow. The only thing that crosses from one chain to the other is a cryptographic proof that a specific event occurred.



No asset bridge — only cryptographic proof of a source-chain event crosses the boundary.

Figure 1. Vouch's two-chain layout. Attestcoin sits between Sepolia (payments) and Creditcoin CC3 (loyalty state and rewards); no asset crosses the boundary, only proof.

What customers and merchants can do

- **Shop from merchant menus** — choose named items with prices and available stock, then pay directly to the merchant's Sepolia address.
- **Prove a payment and earn progress** — proof readiness is tracked in an Activity view; claiming on Creditcoin produces an on-chain receipt and stars, and a first verified purchase mints a Benefit Pass.
- **Complete quests** — qualifying purchases earn merchant-funded CTC cashback, gated by item requirements and account level.

- **Join community campaigns** — qualifying purchases advance a shared target; once it is reached, participants can claim rewards, subject to available funding.
- **Unlock app cashback** — higher levels qualify for a percentage of verified spend, paid from a separate CTC treasury.
- **Redeem surplus stars** — stars above the 700-star Level 5 line swap for CTC at 100 stars = 10 CTC from a dedicated treasury; the level itself never drops.
- **Run a storefront** — merchants get a console for registration, menu and stock management, quests, and funded campaigns.

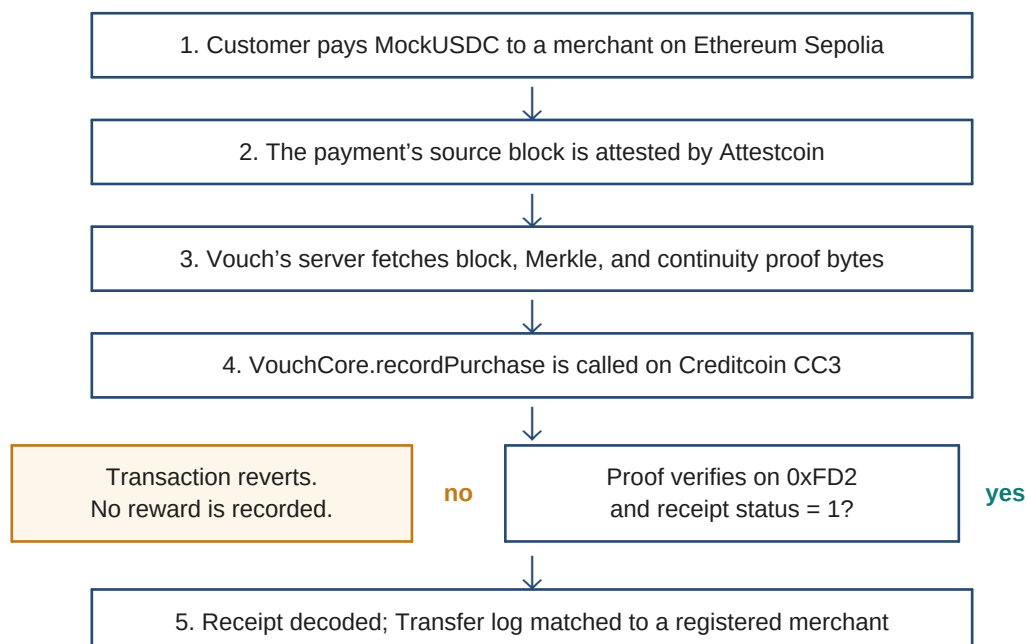
3. Attestcoin as the Trust Primitive

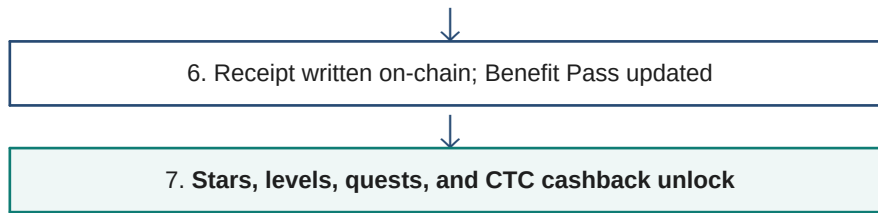
The mechanism that makes the model in Section 2 possible — rather than merely aspirational — is Attestcoin: a native attestation protocol running on Creditcoin that lets a smart contract there verify, directly and without an intermediary, that a specific transaction happened on another chain, and what its outcome was. Two on-chain precompiles do the work: one tracks which block heights on a given source chain have been attested, and the other verifies a supplied Merkle and continuity proof against that attested state and decodes the underlying transaction.

This is a meaningfully different trust model from a typical bridge. A bridge usually moves value by locking an asset on one chain and minting a representative claim on another, which makes the bridge — or its validator set — something users must trust with custody. Attestcoin moves no value at all. It answers a narrower question — did this transaction happen, and did it succeed — and leaves Creditcoin-side contracts to decide what to do with that answer. In Vouch's case, VouchCore decides to mint a receipt, add stars, and unlock rewards.

How a single purchase becomes a reward

A purchase moves through seven steps end to end: the customer pays MockUSDC to a merchant on Sepolia; that payment's source block is attested by Attestcoin; Vouch's server fetches the resulting proof bytes; VouchCore.recordPurchase is called on Creditcoin CC3 with those bytes; the call verifies the proof and checks that the underlying transaction actually succeeded; if it did, the receipt and an ERC-20 Transfer log are decoded and a receipt is written on-chain, the Benefit Pass is updated, and stars, levels, quests, and CTC cashback unlock. If the proof fails to verify, or the underlying transaction did not succeed, the whole call reverts — there is no partial credit.





Replay protection: each source transaction is identified once by (chain key, block height, tx index) — never by a caller-supplied hash.

Figure 2. Proof-to-reward flow for a single purchase.

The integration in detail

Table 1 lays out how the Sepolia (payment) side and the Creditcoin (verification) side of the integration fit together, including the protocol's price-observation extension discussed further below.

Integration point	Implementation
Source network	Ethereum Sepolia, EVM chain ID 11155111; configured Attestcoin chain key 1
Destination network	Creditcoin CC3 testnet, EVM chain ID 102031
Proof generation	POST /api/proof uses @gluwa/usc-sdk 0.18.0 and proofProvider.service.ProofBuilder.getProof(txHash)
Batch proof	POST /api/proof/batch uses getBatchProof (≤10 txs, ≤1000 blocks) with one shared continuity proof; VouchCore.recordPurchasesBatch verifies once and mints one receipt per payment
Attestation waiter	Proof routes resolve the Sepolia chain key at runtime via get_supported_chains and optionally long-poll with waitUntilHeightAttested (waitMs, capped at 25s) before attempting proof retrieval
Proof service hosts	https://proof-gen-api.cc3-testnet.creditcoin.network, with https://prover.cc3-testnet.creditcoin.network as fallback
Attestation progress	chainInfo.PrecompileChainInfoProvider reads the chain-info precompile at 0x00000000000000000000000000000000FD3
On-chain verification	VouchCore.recordPurchase calls verifyAndEmit on 0x00000000000000000000000000000000FD2
Proof payload	Source block height, encoded transaction bytes, Merkle proof, and continuity proof
Receipt decoding	Vendored EvmV1Decoder decodes transaction fields, receipt status, and ERC-20 event logs
Price source network	Ethereum mainnet, EVM chain ID 1; configured Attestcoin chain key 3
Price pair	Uniswap V2 WCTC (old) / USDT 0x4a4F4fcA1a9B673f9eB23b7EeFe9dFbafd8D8140 (no CTC/USDC V2 pair exists on mainnet)
Price proof	PriceOracle.pushSyncProof verifies a mainnet swap through the same 0x0FD2 precompile and reads the pair's Sync log

Table 1. Attestcoin integration points used by Vouch's proof and verification pipeline.

Two design decisions the proof format forces

1. Inclusion is not success. Attestcoin's precompile proves that a transaction was included in an attested block — not that it succeeded. A reverted Sepolia transaction still produces a valid inclusion proof. VouchCore therefore checks the decoded receipt status itself, requiring it to equal 1, rather than assuming a verified proof implies a successful payment.

2. Replay protection cannot rely on a transaction hash. The decoded transaction fields Attestcoin returns — nonce, gas limit, sender, recipient, value, and calldata — do not include the source transaction's hash. Using a caller-supplied hash for replay protection would be forgeable, since nothing ties it cryptographically to the proof. Vouch instead identifies each source transaction by the triple (chain key, block height, transaction index), where the index is calculated by the verifier itself from the Merkle proof, never supplied by the caller. A given purchase can be claimed exactly once, a property demonstrated in practice in Section 6.

Extending the same primitive to price data

VouchCore is not the only Creditcoin contract that consumes Attestcoin proofs. App-level cashback needs to know CTC's approximate USD price, and rather than trust an oracle operator's word for it, PriceOracle proves it: it reads a Uniswap V2 Sync event from Ethereum mainnet — Attestcoin chain key 3, distinct from Sepolia's chain key 1 — through the same 0x0FD2 verifier and decoder used for payments. Because no CTC/USDC Uniswap V2 pool exists on mainnet or Sepolia, the configured pair is WCTC (old) / USDT; Uniswap V3 pools were ruled out because V3 does not emit a Sync event at all. PriceOracle checks receipt success, that the log came from the configured pair, strictly increasing source heights, non-zero reserves, and configured rate bounds before accepting a new price. Merchant-funded quest and campaign rewards never touch the oracle — they are paid directly in CTC.

Step	Evidence
Source swap (Ethereum)	0xfdcac0146c9106fe200fd2825671ee5e152a0bf69be5c1df6a1a4a97560a9523, block 25938627, txIndex 261
Pair	0x4a4F4fcA1a9B673f9eB23b7EeFe9dFbafd8D8140
0x0FD2.verify(chainKey=3, ...)	true
pushSyncProof (Creditcoin)	0x87f64fafc226a4e25d63e63577efd42a85a40a72a3986fd2f12d80c7cf751705, status 1, gasUsed 371,930
Result	ctcPerUsd ≈ 12.998 CTC / \$1, isManual = false, observedAtHeight = 25938627

Table 2. Live Ethereum-mainnet price proof, pushed onto Creditcoin CC3 on 2026-09-09.

An operational lesson: don't trust the docs for chain-key mapping

Calling `get_supported_chains()` on the chain-info precompile returns chain key 1 mapped to Sepolia and chain key 3 mapped to Ethereum — the reverse of what the SDK's own code comments suggest. Vouch resolves the Sepolia chain key at runtime rather than hard-coding it from documentation. Attestation is also not instantaneous: on 2026-09-08 the latest attested Sepolia height trailed the live chain head by roughly 43 blocks, about nine minutes, which is why Vouch treats a claim as an asynchronous process with real progress shown in the UI, rather than something that resolves the moment a payment is sent.

4. Incentive Design

The reward layer sits on top of the verification described in Section 3: nothing below unlocks unless a purchase has already cleared attestation, on-chain verification, and receipt-status checks.

Every verified purchase earns stars at a default rate of 10 stars per whole demo dollar spent, computed proportionally in the payment token's six-decimal units with integer rounding. Stars are not purely a spend counter, though: daily check-ins earn stars as well, so a customer's level reflects engagement, not only wallet size.

Level	Cumulative stars required	Default app cashback
1	0	0%
2	50	2%
3	150	4%
4	350	7%
5	700	10%

Table 3. Level thresholds and default app cashback.

Quest eligibility and the app-cashback rate are both evaluated using the level a customer held before the purchase being processed, not after. A purchase that pushes a customer from level 1 to level 2 is still rewarded at level 1's rate; the new level's benefits apply starting with the next purchase, so a purchase never retroactively changes its own reward.

Reward accounting is kept in three separate pools. Merchant-funded quest and campaign rewards are paid directly in CTC by the merchant and never touch the price oracle. App-level cashback is a percentage of verified spend, paid from a separate app treasury, and requires both a fresh oracle price and sufficient treasury funds — if funds are short, a customer receives a partial credit or none, rather than an IOU.

Past the 700-star Level 5 line, surplus stars become spendable: StarRedeem swaps them for CTC at 100 stars = 10 CTC from its own dedicated treasury (2,000 CTC at deployment). Redemption is tracked in the redeem contract's own ledger against the same floor, so a collector's stars — and therefore level — never drop, and VouchCore itself needed no changes.

5. Contracts and Technology Stack

The customer- and merchant-facing app is a Next.js 15 / React 19 / TypeScript frontend using wagmi, viem, and TanStack Query for wallet and chain state. The server-side proof route uses Gluwa's USC SDK together with ethers. Contracts are written in Solidity 0.8.28, built and tested with Foundry, use OpenZeppelin for standard primitives, and target the London EVM configuration — a constraint discussed further in Section 6.

Contract	Responsibility
CommerceRegistry	Storefront ownership and unique Sepolia payout addresses
Catalog	Named items, prices, stock, and purchase counters
VouchCore	Proof verification, receipts, profiles, check-ins, and reward coordination
BenefitPass	Non-transferable ERC-721 with dynamic on-chain SVG and metadata
QuestManager	Purchase requirements, level gates, and quest completion
RewardVault	Merchant quest funding and cashback withdrawals
MilestoneManager	Community targets, participation, budgets, and claims
PriceOracle	Proven Uniswap V2 Sync reserves from Ethereum mainnet (WCTC/USDT)
AppCashback	Level-based percentage rewards from the app treasury
StarRedeem	Surplus stars above 700 swapped for CTC at 100 stars = 10 CTC
MockUSDC	Six-decimal Sepolia demo token with an open faucet

Table 4. On-chain contracts and their responsibilities.

6. Security, Testing, and Live Validation

92 tests pass across five Foundry suites, covering payment validation, replay rejection, star and level progression, Benefit Pass rendering, quests, catalog stock, community campaigns, oracle behavior, cashback accounting, batch claims, and star redemption.

That coverage is honest about two different things it is proving. One test file, `Decoder.t.sol`, decodes real recorded Attestcoin proof bytes from an actual earlier Sepolia transaction, exercising the real decoder against real data. The current MockUSDC purchase and oracle test suites, by contrast, use encoded fixtures and a mock verifier to exercise the application's logic in isolation — a valid and necessary layer of testing, but not itself a live end-to-end test of the current deployment.

Live proof-to-reward validation

Separately from the Foundry suites, the full proof-to-reward loop was run once against live Sepolia and Creditcoin CC3 infrastructure, shown in Table 5. This run used the project's earlier ETH-payment iteration — a flat 50 XP plus 200,000 XP per ETH, rather than the current MockUSDC star formula — but exercises the identical Attestcoin proof, verification, and replay-protection plumbing that the current deployment relies on.

Step	Evidence
Payment (Sepolia)	0x594450946eb2cd34d55d5255fba80c2aac59e5f6a658f4e4626a3c2f2a635cae, block 11658823, 0.002 ETH
Attestation wait	440s (~7.3 min) from payment to servable proof
Proof	height 11658823, txIndex 74
Claim (Creditcoin)	0x89b303d7cb103874211f671847c9bd6f0d3930e495a72abf6ade684387b5f595, status 1, gasUsed 574,013
Result	Receipt #1, 450 XP, Benefit Pass #1 minted, quest 1 progress 1/2
Replay	A second submission of the same proof is rejected on-chain with <code>PurchaseAlreadyClaimed(0x720b3ff2...)</code>

Table 5. End-to-end proof-to-reward validation against live infrastructure, 2026-09-08 (earlier ETH-payment iteration).

The replay check in that run is worth pulling out on its own: submitting the exact same proof a second time was rejected on-chain with `PurchaseAlreadyClaimed`, rather than silently succeeding or double-crediting the account — a direct, on-chain demonstration of the (chain key, height, transaction index) identity scheme described in Section 3.

Verifying a purchase is not free: `recordPurchase` costs roughly 575,000 gas on Creditcoin, the large majority of it spent inside the `0x0FD2` verification call itself. Gas estimation on CC3 testnet has also proven unreliable in practice — three contract-wiring calls failed outright during initial deployment because their estimated gas equaled their gas limit exactly. Vouch's deploy scripts now request estimates with a 300% multiplier, and its legacy CLI claim helper doubles its own estimate, for the same reason.

Why the EVM target is pinned to London

CC3 testnet block headers omit fields that newer EVM versions expect — `mixHash`, `withdrawalsRoot`, and `excessBlobGas` among them. Targeting Cancun or Shanghai causes Foundry's simulation to fail outright, and a Cancun target risks emitting opcodes (`MCOPY`, `TSTORE`) the chain may not implement, so `foundry.toml` pins `evm_version = "london"`. Even with that pin, `forge` script still logs a harmless-looking “missing field `mixHash`” warning while polling receipts; the transactions land regardless, and results are best verified by querying the chain directly rather than trusting the script's own summary, which can mislabel transactions when receipt polling stumbles.

7. Known Limitations

Vouch is explicit about what it has not solved yet. As a testnet hackathon prototype, it carries the following constraints:

- **Administrative controls.** Protocol owners can currently change reward settings and oracle configuration; these parameters are not yet behind a timelock or governance process.
- **Inventory timing.** Stock is decremented at claim time, not payment time, so a stale or sold-out item can leave a verified, rewarded payment with no corresponding catalog sale. Selecting an item in the app does not itself prove physical fulfillment.
- **Asynchronous infrastructure.** Claims depend on source-chain attestation, prover availability, and RPC access, all outside Vouch's control; pending-payment tracking currently lives in browser `localStorage` rather than a server-side queue.
- **Scale.** The interface currently enumerates contract records directly rather than through a dedicated indexer.
- **A legacy CLI.** A command-line claim helper (`scripts/claim.mjs`) still submits correct claim arguments, but its final profile formatter expects older ETH/XP fields and can fail after a successful submission; the browser claim flow is the supported path for the current demo.

8. Roadmap

Three concrete next steps follow directly from Sections 6 and 7: a recorded, live end-to-end demonstration of the current MockUSDC proof-to-reward flow specifically (the validated run in Section 6 predates the MockUSDC iteration); more robust pending-payment recovery than browser local storage; and an inventory reservation strategy that closes the payment-to-claim timing gap described in Section 7.

Exploring a New Primitive with Attestcoin

Strip away the coffee shops and Benefit Pass artwork, and what Vouch is really testing is narrower and more general: can a smart contract on one chain treat a verified event on a completely different chain as a first-class input — not a bridged asset, not a centralized attestation from a single company, but a cryptographic proof it can check itself? Attestcoin's answer is a native, precompile-level yes, and loyalty rewards are simply the first application layered on top of it.

That is the sense in which this project explores a new primitive with Attestcoin: proof-gated, cross-chain application state. Loyalty stars, levels, and cashback are one instance of that primitive. The same shape of problem — reward, unlock, or grant access based on an event that happened somewhere else, without custody, bridging, or a trusted intermediary in between — generalizes well beyond a single storefront.

Appendix A. Current Testnet Deployment

Creditcoin CC3 testnet, chain id 102031. Recorded, non-custodial contract addresses; not a guarantee of current availability.

Contract	Address
CommerceRegistry	0x4491522938510A999ad28D145BE3782939D6cd43
BenefitPass	0xE1f8754bF3E384f2e42c3c7de8bDc73956451434
VouchCore	0xa208b46d4016D799799591A41500E3f3b44549A0
QuestManager	0x19B1aA6eAb81cBC6d742A966633a9F867b31F7D1
RewardVault	0xD94278AF61cB016CEc7Be98a25E7b0ecf01D61ac
MilestoneManager	0xD295ADCce48f68B6b2369521a2eFc6f4260305a4
Catalog	0x1eb050Db90c64Ca67C2F424414b7C193E7040Bb9
PriceOracle	0xE705Ee700Cd619e98751ca1D072B87931Ce5e81D
AppCashback	0x9E7063023e65CD1c593C7d0397C3F7692Af92700
StarRedeem	0xfdb448ccf3eb859721192b108e344c386c22501f
PaymentToken (Sepolia MockUSDC)	0xBcb107E49F78C9dBa122eA31137B6ED903F65AeC

VouchCore.CHAIN_KEY is 1 (Sepolia); PriceOracle.CHAIN_KEY is 3 (Ethereum). Both bind their verifier to the same live 0x0FD2 precompile. The app-cashback treasury holds 2,020 CTC, after a 2,000 CTC top-up on top of the original 20 CTC seed. StarRedeem (floor 700 stars, 0.1 CTC per star) holds a separate 2,000 CTC redemption treasury.

Appendix B. Attestcoin Protocol Constants

Constants and behaviors verified live against CC3 testnet by the Vouch team, rather than taken solely from protocol documentation.

Thing	Value
Creditcoin CC3 testnet chainId	102031
Creditcoin RPC	https://rpc.cc3-testnet.creditcoin.network
Block prover precompile	0x00FD2
Chain info precompile	0x00FD3
Prover service	https://prover.cc3-testnet.creditcoin.network
Proof endpoint	GET /api/v1/proof-by-tx/{chainKey}/{txHash}
Sepolia chainKey	1 (payments / VouchCore)
Ethereum mainnet chainKey	3 (Uniswap V2 Sync / PriceOracle)
Uniswap V2 price pair	WCTC (old) / USDT 0x4a4F4fcA1a9B673f9eB23b7EeFe9dFbafD8D8140
Solidity source	@gluwa/usc-contracts@0.2.0, vendored into contracts/src/vendor/
TS SDK	@gluwa/usc-sdk@0.18.0 (ethers v6 peer dep)

Appendix C. Earlier ETH-Payment Deployment

Historical reference only. These addresses belong to Vouch's first, ETH-denominated iteration and are not part of the current MockUSDC deployment listed in Appendix A.

Contract	Address
CommerceRegistry	0xbB54d51f8bDA5B9bBb0364e8e555a5623850f66d
BenefitPass	0xBcb107E49F78C9dBa122eA31137B6ED903F65AeC
VouchCore	0x5F356e04bB11651eD083Ca9b6abF19fe4bE26855
QuestManager	0x361E1e7E26267207185d96e617Aae44d7F93d517
RewardVault	0x3a1081e9B494Af63d254cf64f90189ea85481f9E

Prepared from the [dun999/vouch repository \(README.md and ATTESTCOIN.md\)](#) as of September 2026. This is a community/hackathon technical document, not an audited security review, and not an offer of any security or financial instrument.